

Projekthandout - Nom Nom Now

1. Projektname

Nom Nom Now

Nom Nom Now ist eine Rezept- und Essensplanungs-App. Nutzerinnen und Nutzer können Rezepte entdecken, erstellen, bearbeiten und löschen, einen Wochenplan pflegen und daraus Einkaufslisten generieren.

Repositorys:

- Frontend: [git@github.com:Nom-Nom-Now/nom-nom-now-frontend.git](https://github.com/Nom-Nom-Now/nom-nom-now-frontend.git)
- Backend: [git@github.com:Nom-Nom-Now/nom-nom-now-backend.git](https://github.com/Nom-Nom-Now/nom-nom-now-backend.git)
- Scrum-Board: <https://nomnomnow.atlassian.net/jira/software/projects/NNN/boards/1>
- Weitere Dokumentation/Blog/Zusammenfassung: <https://github.com/Nom-Nom-Now/docs>

2. Projektvision und Ziel

Ziel des Projekts ist eine alltagstaugliche Web-App für Essensplanung. Der Kernnutzen liegt darin, Rezepte, Wochenplanung und Einkaufsvorbereitung in einem zusammenhängenden System abzubilden:

- Rezepte können mit Zutaten, Zubereitung, Kategorien und optionalem Bild gepflegt werden.
- Rezepte können über eine Liste durchsucht und in Detailansichten geöffnet werden.
- Ein Wochenplan ordnet Rezepte einzelnen Tagen zu.
- Aus einem Wochenplan können Einkaufslisten erzeugt und später wieder geöffnet oder gelöscht werden.
- Authentifizierung läuft produktiv über Google-OAuth2; lokal gibt es ein Dev-Profil mit Testnutzer.

3. Aufwandsstatistiken

3.1 Arbeitsstunden pro Person

Die folgenden Arbeitsstunden fassen die im Team erfassten Aufwände zusammen.

Person	Arbeitsstunden	Hauptbeitrag	Quelle/Kommentar
Rafael Till	65	CI/CD und Backend-Entwicklung	Git-Autor in Frontend und Backend sichtbar: rafiistcool
Silas Scholler	54	Wochenaufgaben und Frontend-Entwicklung	Git-Autor in Frontend und Backend sichtbar: SilasSch
Tino Wolter	63	Frontend-Entwicklung, Frontend-Architektur, Mockups und Reviews	Git-Autor im Frontend und Backend sichtbar: tinow04
Robin Fischer	58	Frontend-Entwicklung, Mockups, Backend-Entwicklung, Wochenaufgaben	Git-Autor im Frontend und Backend sichtbar: Robin0106 und Paircode mit Rafael im Backend
Dylan O'Reilly	43	Frontend-Entwicklung und Projektmanagement	Git-Autor in Frontend und Backend sichtbar: d1111n , mündliche Prüfung im 3. Semester

3.2 Arbeitsstunden pro Workflow

Workflow	Arbeitsstunden	Kommentar
Requirement Analysis	10	Anforderungen, User Stories, Akzeptanzkriterien
Project Management	50	Sprintplanung, Boardpflege, Reviews, Koordination
Architecture & Design	20	Frontend-/Backend-Struktur, Datenmodell, API-Verträge
Frontend Development	50	Vue-App, Feature-Module, UI, i18n
Backend Development	50	Spring-Boot-API, Services, Security, Datenbankzugriff
Database Design & Migration	15	PostgreSQL-Schema, Flyway-Migrationen
Testing & Quality Assurance	20	Unit-/Service-/Controller-Tests, Linting, Type-Checks
CI/CD & Deployment	15	GitHub Actions, Docker-Images, Server-Deployment
Documentation & Presentation	25	README, Handout, Folien, Abschlussdokumentation

Die Stunden sind bei den Stunden pro Person teilweise doppelt eingerechnet, wenn mehrere Personen an einem Workflow gearbeitet haben.

3.3 Arbeitsstunden pro Phase

Phase	Arbeitsstunden	Schwerpunkt
Inception	45	Projektidee, Vision, erste Anforderungen, Risiken
Elaboration	31	Architekturentscheidungen, SRS/SAD, Datenmodell, Prototypen
Construction	130	Implementierung der Kernfeatures, Tests, CI/CD
Transition	47	Stabilisierung, Demo, Dokumentation, Deployment

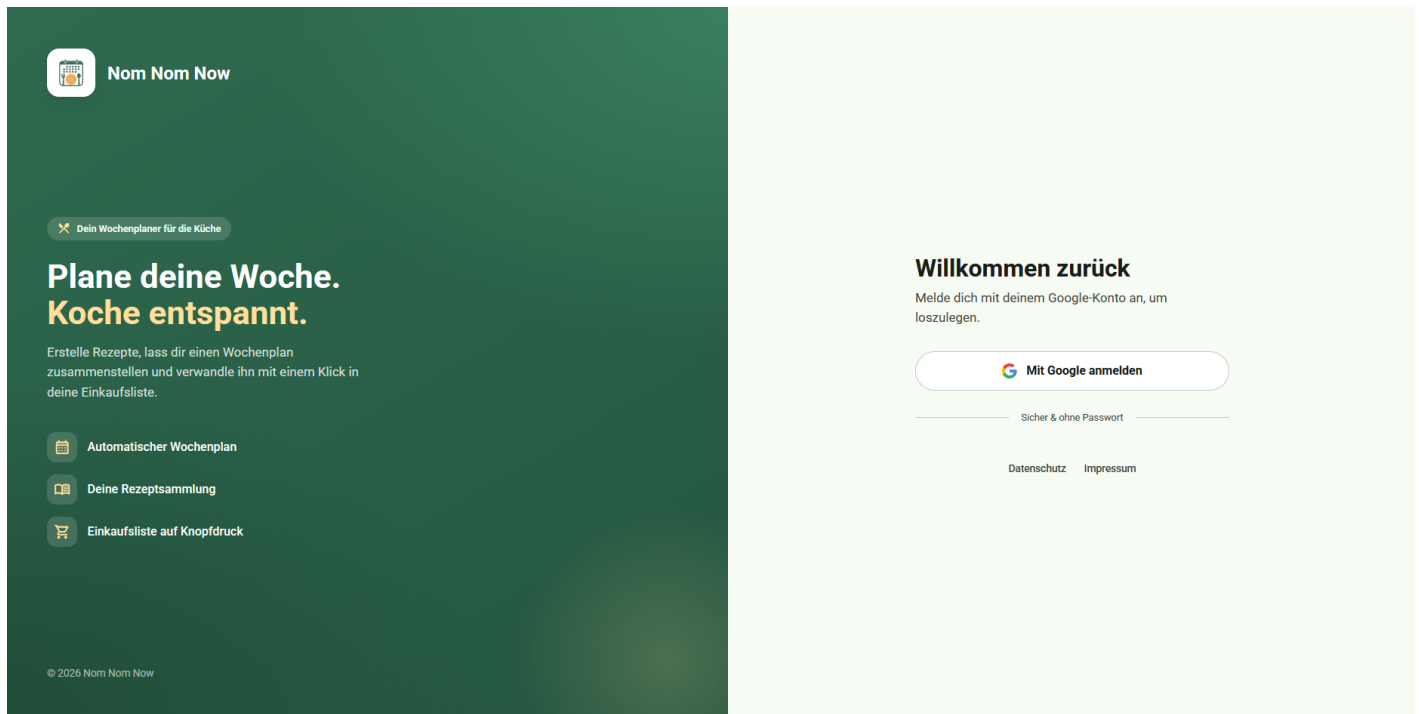
4. Demo-Highlights

4.1 Login und Startseite

Beschreibung:

- Produktiv: Google-OAuth2-Login.
- Lokal: Dev-Profil mit automatisch angemeldetem Testnutzer.
- Die App nutzt eine Shell mit Navigation und Seitentitel. Login wird als Vollbildseite ohne Shell angezeigt.

Screenshot:



4.2 Rezepte erstellen

Beschreibung:

- Mehrstufiger Rezept-Wizard.
- Schritte: Zutaten, Zubereitung, Kategorien, Bild, Vorschau.
- Backend validiert Requests und speichert Rezept, Zutaten, Komponenten, Kategorien und optional Bilddaten.

Screenshot:

Startseite

Rezepte

Plan

Einkaufslisten

Deutsch

Rezept erstellen

Petr Lustig

1 Zutaten

2 Zubereitung

3 Kategorien

4 Foto

5 Vorschau

Name

z.B. Pizza Hawaii

Pizza

Portionen

Das Rezept ist für

1

Personen

Preis

Gesamtpreis (€)

10

≈ 10.00 € pro Person

Zutaten und Mengenangaben

Zutat

Mehl

Menge

500

Einheit

g

↑

↓

🗑

Zutat

Wasser

Menge

200

Einheit

ml

↑

↓

🗑

+ Zutat hinzufügen

Startseite

Rezepte

Plan

Einkaufslisten

Deutsch

Rezept erstellen

Petr Lustig

1 Zutaten

2 Zubereitung

3 Kategorien

4 Foto

5 Vorschau

Zubereitung

Beschreibe die Schritte zur Zubereitung deines Rezepts

Teig machen ...

Zubereitungszeit

Stunden

1

Minuten

0

Startseite

Rezepte

Plan

Einkaufslisten

Deutsch

Rezept erstellen

Petr Lustig

1 Zutaten

2 Zubereitung

3 Kategorien

4 Foto

5 Vorschau

Wähle deine Kategorien

Ganzjährig

Italienisch

Hauptgericht

Backen

Saisonal	Hauptzutat	Herkunft	Gang
Unterkategorien auswählen	Unterkategorien auswählen	Unterkategorien auswählen	Unterkategorien auswählen
Sonstiges	Getränke	Zubereitungsart	Ernährungsweise
Unterkategorien auswählen	Unterkategorien auswählen	Unterkategorien auswählen	Unterkategorien auswählen

Rezept erstellen

Petr Lustig

1 Zutaten

2 Zubereitung

3 Kategorien

4 Foto

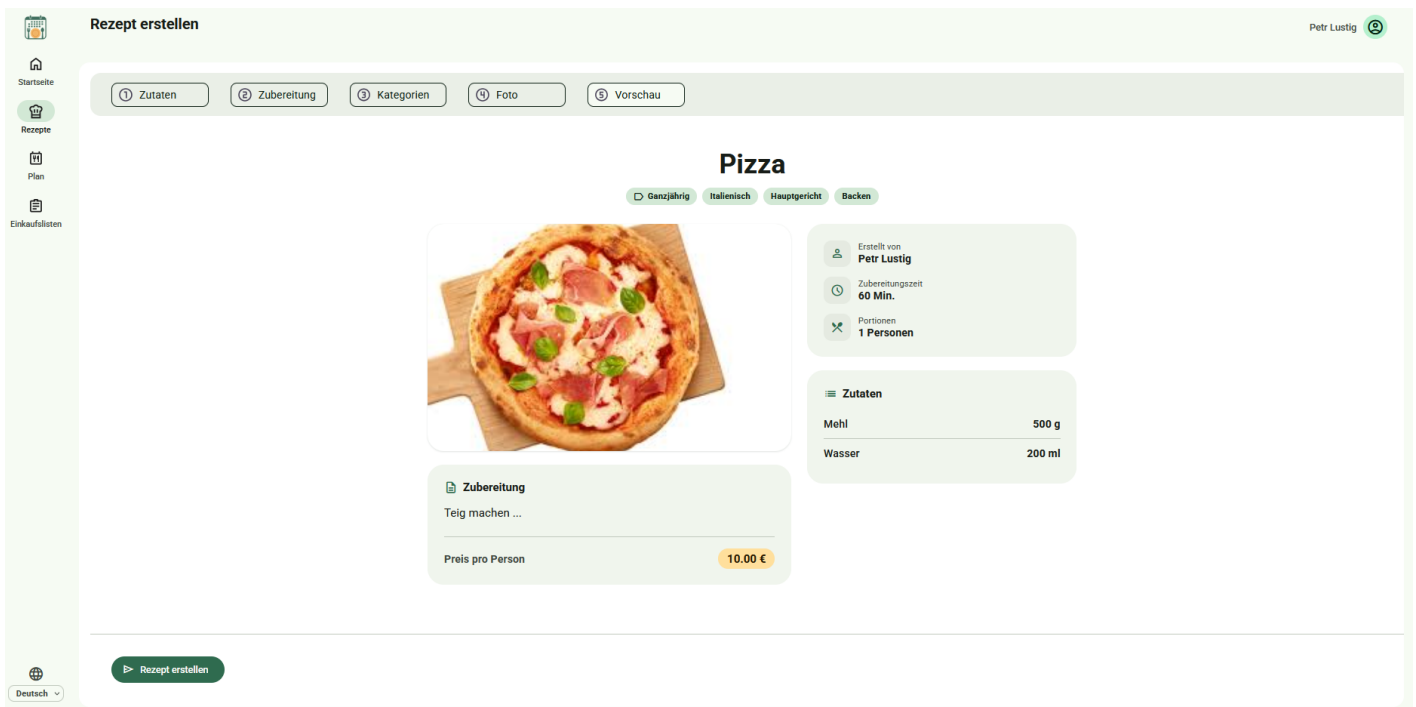
5 Vorschau

Foto



Entfernen

Foto auswählen

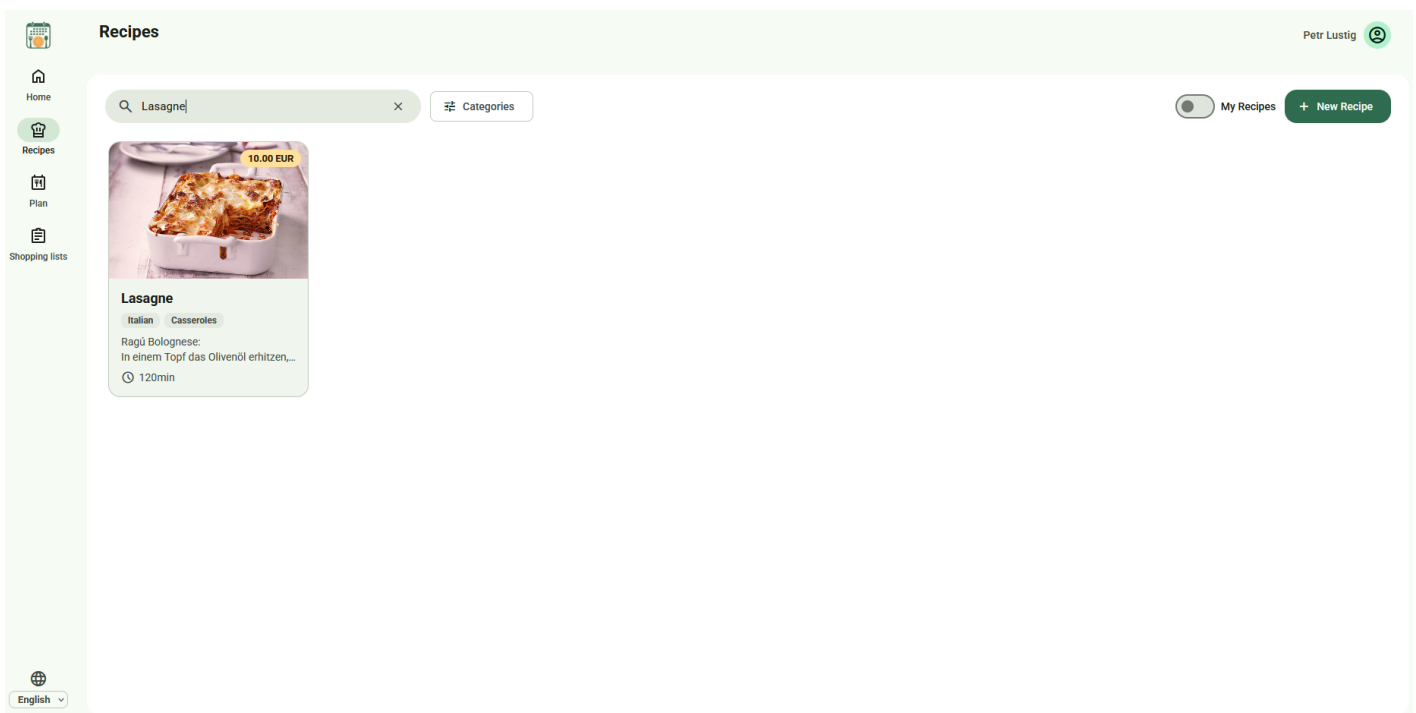


4.3 Rezepte suchen, ansehen, bearbeiten und löschen

Beschreibung:

- Rezeptliste mit Such-/Browse-Flow.
- Detailansicht für einzelne Rezepte.
- Bearbeitungsflow mit eigenem Store und Service.
- Löschen ist als geschützter Owner-Flow im Backend umgesetzt.

Suchfunktion:



Detailansicht:



Pizza Diavolo

📁 Italienisch

🍳 Backen

🍽 Mittagessen

🍷 Abendessen

Hauptgericht

🕒 60min 🍴 1 Portionen

Teig machen.

Tomatensoße drauf.

Pizza belegen.

Pizza in Ofen.

Ofen an.



Rezept von: Petr Lustig

☰ Zutaten

8 Zutaten

Mehl	400 g
Olivenöl	4 EL
Paprika	50 g
Peperoni	20 g
Salami	50 g
Salz	2 TL

Bearbeiten:

The 'Edit Recipe' interface for a 'Pizza' recipe. The title 'Pizza' is centered at the top. Below it is a large image of a pizza. To the right of the image, there are three sections: 'Created by Petr Lustig', 'Preparation time 60 Min.', and 'Servings 1 people'. Below these is the 'Ingredients' section, which lists 'Mehl 500 g' and 'Wasser 200 ml'. Below the ingredients is the 'Preparation' section, which contains the text 'Teig machen ..' and 'Price per person 10.00 €'. At the bottom of the interface are two buttons: 'Cancel' and 'Save changes'. The left sidebar contains navigation links: Home, Recipes, Plan, and Shopping lists. The top right corner shows the user's name 'Petr Lustig' and a profile icon. The bottom left corner shows a language dropdown set to 'English'.

Edit Recipe

Petr Lustig

① Ingredients ② Preparation ③ Categories ④ Image ⑤ Preview

Pizza

Created by **Petr Lustig**

Preparation time **60 Min.**

Servings **1 people**

Ingredients

Mehl	500 g
Wasser	200 ml

Preparation

Teig machen ..

Price per person **10.00 €**

Cancel Save changes

English

Löschen:

The 'Recipes' interface shows a list of recipes. The 'Pizza' recipe is selected, and a confirmation dialog is displayed over it. The dialog asks 'Are you sure you want to permanently delete this recipe?' and has two buttons: 'Cancel' and 'Confirm'. The background is dimmed. The left sidebar contains navigation links: Home, Recipes, Plan, and Shopping lists. The top right corner shows the user's name 'Petr Lustig' and a profile icon. The bottom left corner shows a language dropdown set to 'English'.

Recipes

Petr Lustig

← Back Edit Delete

Pizza

Italian All season Bake Main course

Are you sure you want to permanently delete this recipe?

Cancel Confirm

Created by **Petr Lustig**

Preparation time **60min**

Servings **1 people**

Ingredients

Mehl	500 g
Wasser	200 ml

Preparation

Teig machen ...

Price per person **10.00 EUR**

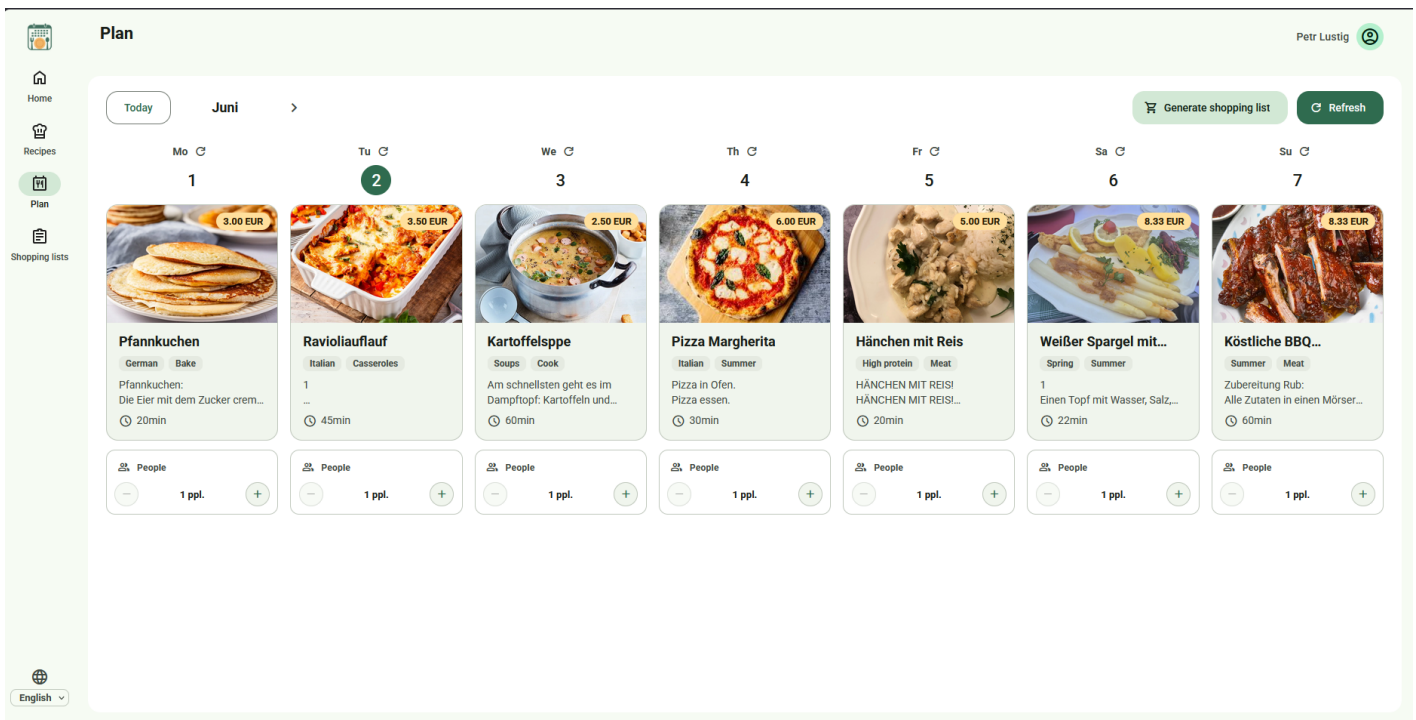
English

4.4 Wochenplan

Beschreibung:

- Wochenplan unter `/plan`.
- Backend-Endpunkte für Wochenplan laden, speichern und einzelne Tage neu belegen.
- Pro Nutzer und Datum ist ein Planeintrag eindeutig.

Screenshot:

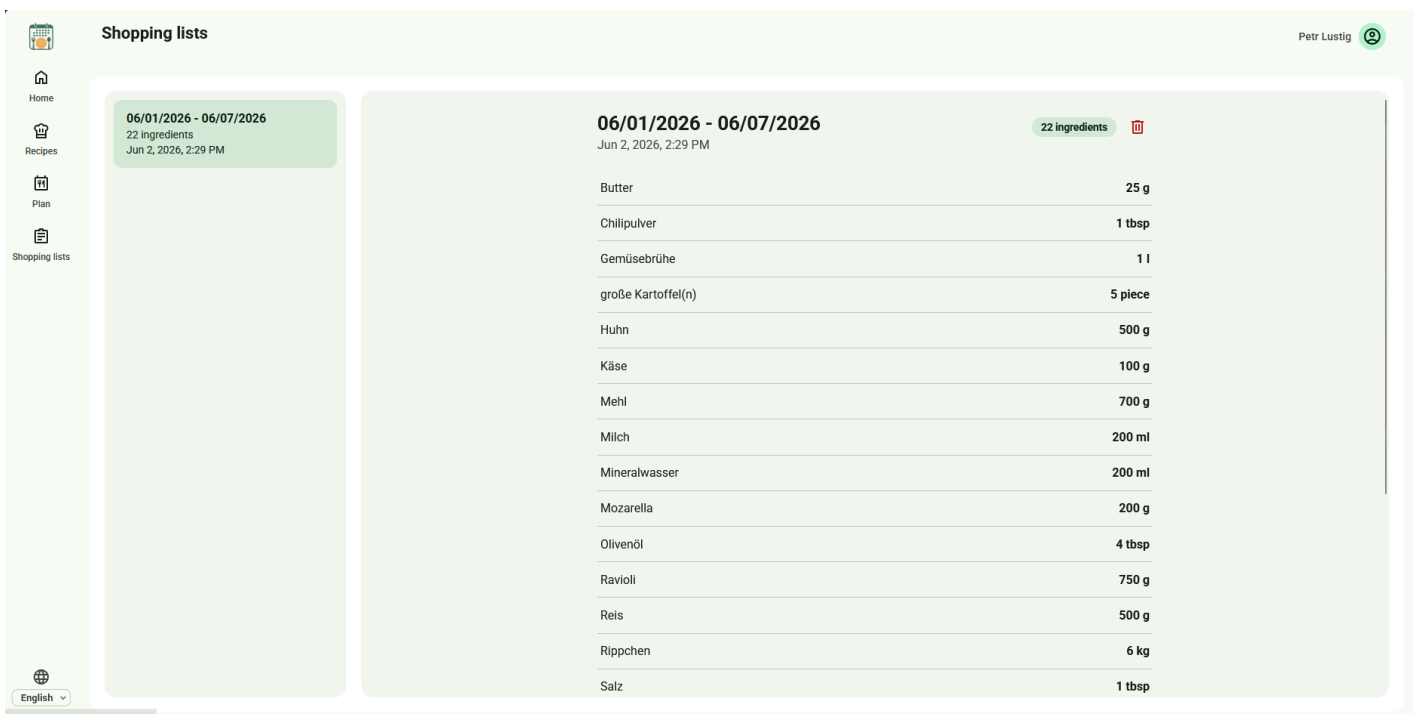


4.5 Einkaufslisten

Beschreibung:

- Einkaufslisten werden aus Wochenplänen generiert.
- Gespeicherte Einkaufslisten können wieder geöffnet werden.
- Einkaufslisten sind nutzerbezogen.
- Löschen ist owner-scoped und im Frontend mit Bestätigungsdialog abgesichert.

Screenshot:



5. Projekt-Highlights

5.1 Architektur

Nom Nom Now ist als getrennte Frontend-/Backend-Webanwendung aufgebaut.

Browser

-> Vue 3 SPA

-> Feature-Komponenten

- > Pinia Stores
- > Services / DTO-Mapping
- > HTTP-Requests
- > Spring Boot REST API
- > Controller
- > Services
- > Repositories
- > PostgreSQL

Frontend:

- Vue 3 Single Page Application mit Composition API.
- Feature-basierte Struktur unter `src/feature/`.
- App-Shell-Pattern mit Navigation, Header und `<router-view />`.
- Vue Router mit Routen für Login, Home, Plan, Rezepte, Rezeptbearbeitung und Einkaufslisten.
- Pinia für reaktiven Feature-State.
- Services kapseln API-Kommunikation und Mapping zwischen UI-Modell und Backend-DTOs.
- i18n über `vue-i18n` mit deutschen und englischen Locale-Dateien.
- Material Web Components für UI-Bausteine.

Backend:

- Spring Boot 4 REST API mit Java 25.
- Schichten: Controller, DTOs, Mapper, Services, Repositories, Entities.
- Spring Data JPA für Datenzugriff.
- PostgreSQL als relationale Datenbank.
- Flyway für versionierte Datenbankmigrationen.
- Spring Security mit Google-OAuth2 im Produktivbetrieb.
- Dev-Profil mit lokalem Testnutzer für schnellere Entwicklung ohne Google-Login.

5.2 Architekturentscheidungen und Begründung

Entscheidung	Begründung
Getrennte Frontend-/Backend-Repos	Klare Verantwortlichkeiten, unabhängige CI/CD-Pipelines und getrennte Deployments
Vue 3 + Vite + TypeScript	Schnelle Entwicklung, Typsicherheit, moderne SPA-Struktur
Feature-basierte Frontend-Struktur	Features kapseln Komponenten, Stores, Services und Typen erleichtert parallele Teamarbeit
Spring Boot + Spring Data JPA	Schnelle REST-API-Entwicklung, gute Integration von Persistence, Validation und Security
PostgreSQL + Flyway	Robustes relationales Modell und nachvollziehbare Schema-Evolution
OAuth2 mit Google	Standardisierte Authentifizierung statt eigener Passwortverwaltung
Dev-Profil ohne OAuth2	Lokale Entwicklung und Tests ohne externe Login-Abhängigkeit
Docker + GHCR	Reproduzierbare Builds und deploybare Container-Artefakte

5.3 Angewandte Design Patterns

Pattern/Prinzip	Umsetzung im Projekt
Layered Architecture	Backend trennt Controller, Service, Repository und Entity
Repository Pattern	Spring Data Repositories kapseln Datenbankzugriffe
DTO Pattern	Request-/Response-Klassen trennen API-Vertrag von JPA-Entities
Mapper Pattern	<code>RecipeMapper</code> bildet zwischen Entity und Response DTO ab
Service Layer	Business-Logik liegt in <code>RecipeService</code> , <code>RecipePlanService</code> , <code>ShoppingListService</code> , <code>CategoryService</code>
Feature Module Pattern	Frontend-Features besitzen eigene Komponenten, Stores, Services und Typen
Store Pattern	Pinia Stores verwalten UI- und Feature-State
App Shell Pattern	Gemeinsame Navigation/Header-Struktur mit routenbasiertem Vollbildmodus für Login

6. Tech-Stack

6.1 Frontend

Bereich	Technik
Framework	Vue 3.5
Sprache	TypeScript 5.8
Build Tool	Vite 7
Routing	Vue Router 4
State Management	Pinia 3
Internationalisierung	vue-i18n 11
UI	Material Web Components
Tests	Vitest, Vue Test Utils, jsdom
Linting/Formatierung	ESLint 9, Prettier 3
Deployment	Docker-Multi-Stage-Build, Nginx, GHCR

6.2 Backend

Bereich	Technik
Framework	Spring Boot 4.0.5
Sprache	Java 25
Build Tool	Maven Wrapper
API	Spring Web MVC
Persistence	Spring Data JPA
Datenbank	PostgreSQL
Migrationen	Flyway
Security	Spring Security, OAuth2-Client, Google-Login
API-Dokumentation	springdoc-openapi / Swagger UI
Tests	Spring Boot Test, JUnit, Mockito, Testcontainers
Deployment	Docker-Multi-Stage-Build, GHCR, Docker Compose

6.3 Infrastruktur

- GitHub Actions für CI/CD.
- GitHub Container Registry (ghcr.io) für Frontend- und Backend-Images.
- Docker Compose für PostgreSQL, Flyway, Backend und Frontend.
- Produktiv-Deployment per SSH/SCP auf Server.

7. Datenbankdesign

Die Datenbank liegt in PostgreSQL im Schema **app**. Schemaänderungen werden mit Flyway versioniert. Aktuell liegen 10 Migrationen vor.

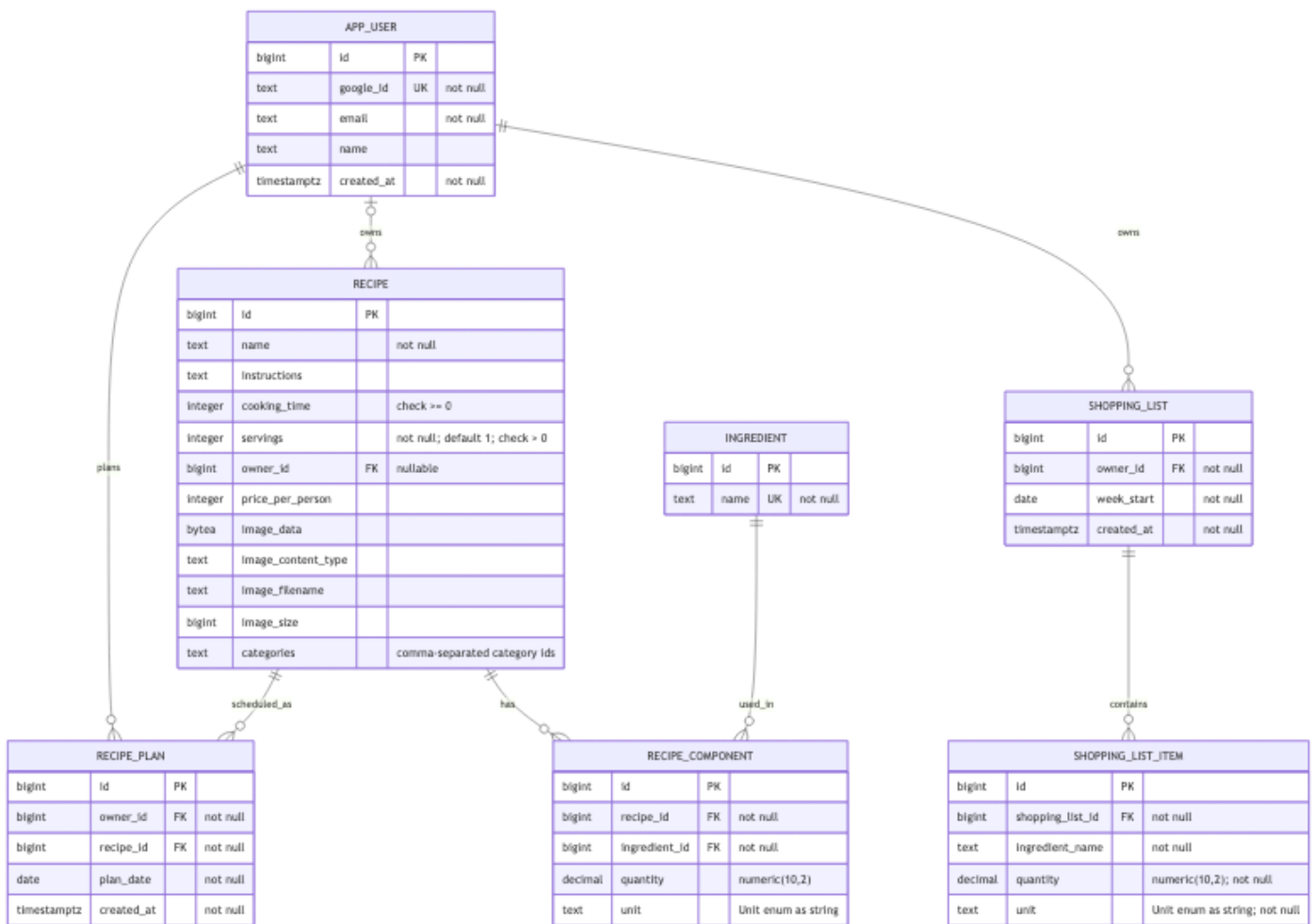
7.1 Zentrale Tabellen

Tabelle	Zweck
<code>app.app_user</code>	Nutzer mit Google-ID, E-Mail, Name und Erstellzeitpunkt
<code>app.recipe</code>	Rezeptstammdaten, u.a. Name, Zubereitung, Kochzeit, Preis-/Bildaten und Owner
<code>app.ingredient</code>	Zutatenstammdaten
<code>app.recipe_component</code>	Zuordnung Rezept - Zutat mit Menge und Einheit
<code>app.category</code>	Kategorien
<code>app.recipe_category</code>	Many-to-many-Zuordnung Rezept - Kategorie
<code>app.recipe_plan</code>	Nutzerbezogene Wochenplan-Einträge
<code>app.shopping_list</code>	Nutzerbezogene Einkaufslisten
<code>app.shopping_list_item</code>	Positionen einer Einkaufsliste

7.2 Beziehungen und Constraints

- Rezepte gehören optional/eindeutig zu einem Owner (`recipe.owner_id -> app_user.id`).
- Rezeptkomponenten referenzieren Rezept und Zutat; beim Löschen eines Rezepts werden Komponenten per Cascade entfernt.
- Rezept-Kategorien sind als Many-to-many-Tabelle modelliert.
- Wochenpläne referenzieren Owner und Rezept.
- Pro Owner und Datum gibt es im Wochenplan nur einen Eintrag (`UNIQUE (owner_id, plan_date)`).

- Einkaufslisten und Einkaufslistenpositionen sind per Fremdschlüssel verbunden; Items werden beim Löschen der Liste per Cascade entfernt.
- Datenbankindizes existieren u.a. für Rezeptkomponenten, Kategorien, Rezept-Owner, Wochenplan nach Owner/Datum und Einkaufslisten nach Owner/Erstellzeitpunkt.



8. Testing und Qualitätssicherung

8.1 Backend-Tests

Abgedeckte Bereiche:

- Application Context
- Controller-Verhalten für Rezepte, Einkaufslisten und Auth
- Service-Logik für Rezepte, Wochenpläne und Einkaufslisten
- Demo-Daten-Seeding
- Current-User-Auflösung

CI-Befehl:

```
./mvnw -B verify
```

8.2 Frontend-Tests

Abgedeckte Bereiche:

- Komponentenverhalten
- Pinia Stores
- API-Services
- DTO-/Kategorie-Mapping
- Auth- und Redirect-Hilfen

CI-Befehle:

```
npm run type-check
npm run lint
npm run test:unit
npm run build
```

8.3 Teststrategie

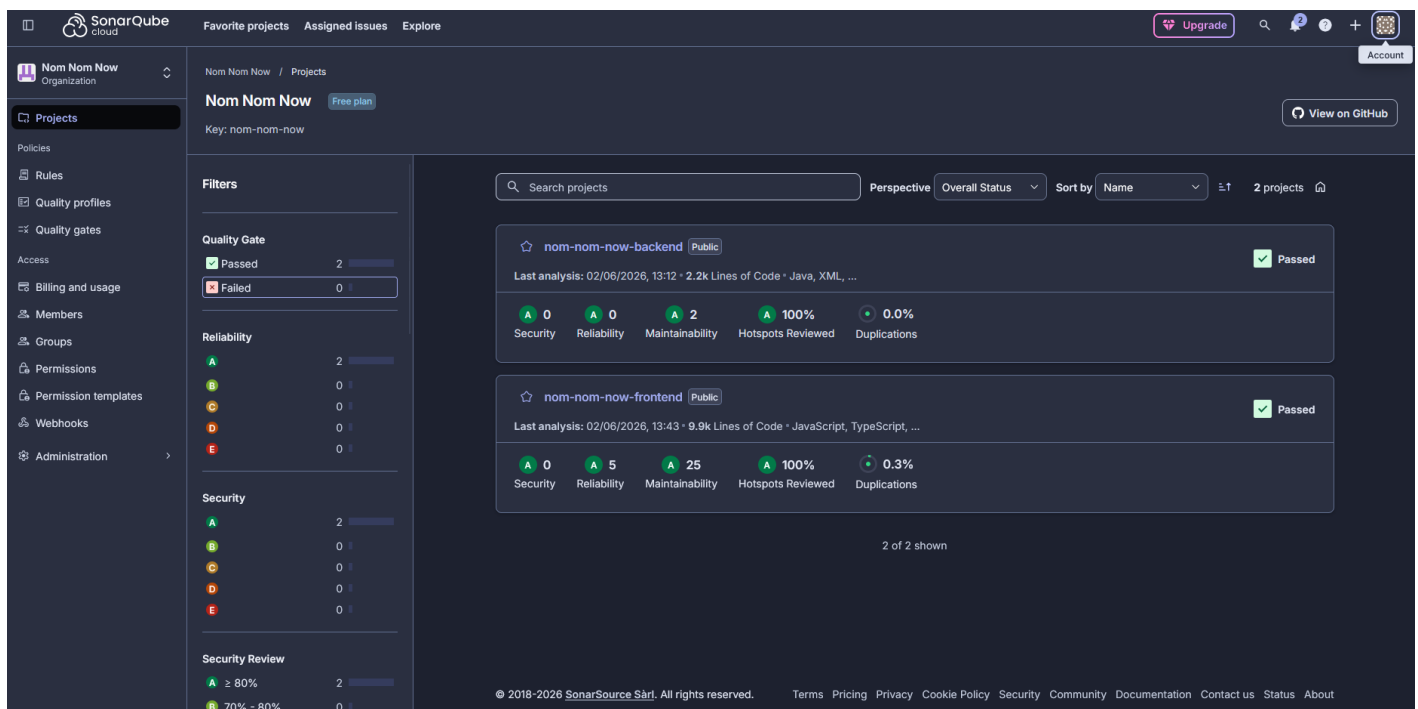
- Backend: Unit-/Service-Tests und Controller-Tests mit Spring-Testumgebung.
- Frontend: Vitest-Tests für Stores, Services und wichtige Komponenten.
- TypeScript Type Checks verhindern viele Integrationsfehler im Frontend.
- ESLint und Prettier sichern Code-Stil und Konsistenz.
- CI läuft bei Pull Requests bzw. Pushes und blockiert fehlerhafte Builds.

8.4 Coverage

Bereich	Tool	Testlauf	Ergebnis	Lines	Statements / Instructions	Branches	Functions / Methods
Frontend	Vitest V8 Coverage	npm run test:coverage	20 Testdateien, 94 Tests, 0 Fehler	75.66% (824/1089)	74.58% (848/1137 Statements)	57.56% (407/707)	68.79% (194/282 Functions)
Backend	JaCoCo 0.8.14	./mvnw test	56 Tests, 0 Fehler	74.93% (562/750)	78.95% (3109/3938 Instructions)	56.37% (115/204)	77.60% (142/183 Methods)

9. Softwarequalität

Angestrebtes 3A in den Metriken über SonarQube:



10. CI/CD-Setup

10.1 Backend CI

Datei: `.github/workflows/backendCi.yml`

- Trigger: Pull Requests auf `main`.
- Setup: Ubuntu Runner, Checkout, Java 25 mit Temurin, Maven Cache.
- Schritt: `./mvnw -B verify`.

10.2 Backend CD

Datei: `.github/workflows/backendCd.yml`

- Trigger: Push auf `main`.
- Docker Buildx baut das Backend-Image.
- Push nach GitHub Container Registry mit Tags `latest` und Kurz-SHA.
- Deployment-Job:

- kopiert `compose.yaml` und Flyway-Migrationen per SSH/SCP auf den Server,
- erzeugt `.env` aus GitHub Secrets,
- startet PostgreSQL,
- führt Flyway-Migrationen aus,
- deployed das Backend per Docker Compose.

10.3 Frontend CI

Datei: `.github/workflows/ci.yml`

- Trigger: Push und Pull Requests auf `main` und `develop`.
- Setup: Node.js 20 mit npm Cache.
- Schritte:
 - `npm ci`
 - `npm run type-check`
 - `npm run lint`
 - `npm run test:unit`
 - `npm run build`

10.4 Frontend CD

Datei: `.github/workflows/frontendCd.yml`

- Trigger: Push auf `main`.
- Docker Buildx baut das Frontend-Image.
- Build-Argument: `VITE_API_BASE_URL=/api`.
- Push nach GitHub Container Registry mit Tags `latest` und Kurz-SHA.
- Deployment per SSH und Docker Compose.

11. Gelerntes im Projekt

- Abstimmung von Frontend-DTOs und Backend-API ist entscheidend, um Integrationsfehler zu vermeiden.
- Authentifizierung sollte für lokale Entwicklung bewusst vereinfacht werden, ohne die produktive Sicherheit zu entfernen.
- Feature-basierte Struktur hilft bei wachsenden Vue-Anwendungen.
- Gute Tests für Services und Stores fangen viele Fehler ab, bevor sie in der UI sichtbar werden.
- Regelmäßige Reviews und Pair Programming fördern den Wissensaustausch und die Codequalität.